

BUIDL CTC 2026 FALL — CREDITCOIN HACKATHON SUBMISSION

SafeRoot Policy

One Safe approval on Ethereum. Trustless verification via Creditcoin's Attestcoin oracle. Execution that refuses anything that wasn't exactly what was approved. No bridge, no new signers.

Live app: frontend-production-78b3.up.railway.app

Repository: github.com/Vamp-Labs/Saferoot

Demo video: youtu.be/6N_fVw4CcgM

Cross-chain treasury authority means trusting something new

Safe multisig is the standard for DAO and protocol treasuries — but its authority stops at the chain it lives on. The moment a team wants an approved action to affect another chain, every common answer introduces a new party the treasury has to trust.

Bridge the assets

A bridge contract or its operators become a single point of failure holding the funds. Bridges are, by dollar value, the most exploited category of smart contract in crypto history.

Second signer set

Deploying a separate multisig on the destination chain doubles key-management surface and governance overhead — now two committees must stay honest and in sync.

Centralized relayer

A single company or server sits between a governance decision and the funds moving — exactly the kind of intermediary decentralized treasuries exist to avoid.

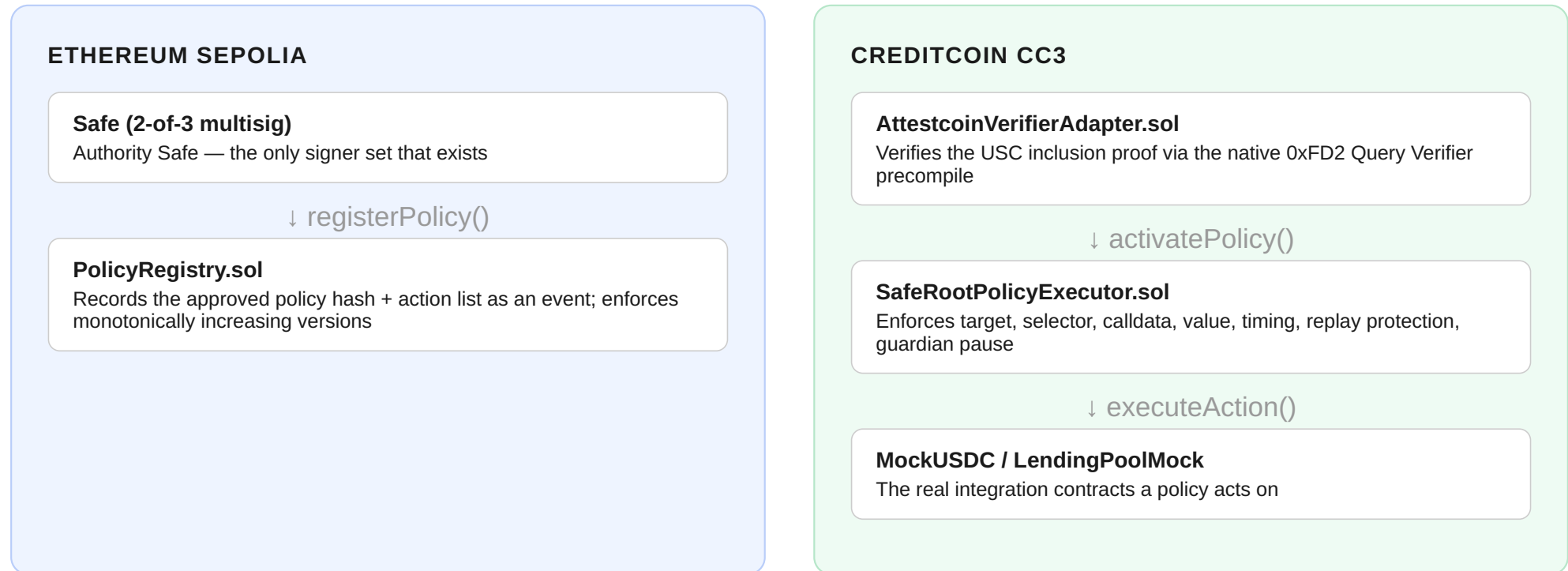
None of these actually need to exist. A DAO doesn't need to move assets cross-chain to act cross-chain — it needs the destination chain to be convinced, cryptographically, that an approval genuinely happened. That's a proof problem, not a bridging problem.

Approve once. Prove it. Verify it natively. Enforce it exactly.



The result: a DAO's Ethereum Safe becomes the source of authority for actions on Creditcoin, with zero new trusted parties in between.

Two chains, one source of authority



Off-chain: a Fastify backend watches the Safe Transaction Service for signature progress, requests Attestcoin proofs via [@gluwa/usc-sdk](#), and submits the on-chain transactions. A Next.js frontend renders the full lifecycle and a live "attempt a tampered execution" control.

A real integration of Attestcoin, not a mock of the interface

- **Attestcoin / USC** is Creditcoin's answer to trustless cross-chain data — SafeRoot integrates it directly (@gluwa/usc-sdk on the backend, @gluwa/asc-contracts proof types on-chain), not a stand-in interface.
- The **native 0xFD2 Query Verifier precompile** is called directly from `AttestcoinVerifierAdapter.sol` — proof verification happens at the chain level, exactly the trust-minimization story Creditcoin's stack is built to enable.
- Building this surfaced a real integration trap worth documenting: Attestcoin's internal **chainKey** for a source chain is **not** the same as its EVM **chainId**. Sepolia's EVM chainId is 11155111; its Attestcoin chainKey is 1.

We hit this as a real production bug: a misconfigured adapter caused every proof verification to revert with `UnsupportedSourceChain` until we root-caused it by decoding the raw revert data byte-by-byte. It's documented here because it's exactly the trap a team using Attestcoin for the first time will hit.

Every guard is enforced on-chain, independent of the caller

GUARD	ENFORCED BY REVERT
Only the registered Safe's policy is honored	SafeMismatch
Only the last-approved version is valid	StalePolicyVersion
Policy must not have expired	PolicyExpired / PolicyAlreadyExpired
An Attestcoin proof can't be reused	ProofAlreadyUsed
Only the trusted Sepolia emitter is accepted	EmitterNotApproved
Target contract must match exactly	TargetMismatch
Function selector must be pre-allowed	FunctionNotAllowed
Calldata must match exactly	CalldataMismatch
Native value can't exceed what was approved	AmountExceedsApproval
An action can't execute before its window or after expiry	NotYetEligible / ActionExpired
An action can't execute twice	ActionAlreadyExecuted
A paused policy blocks execution	ExecutorPaused
Guardian pause is guardian-only	NotGuardian
Reentrancy is blocked	OpenZeppelin ReentrancyGuard

Verified, not claimed

2

real testnets deployed to
(Ethereum Sepolia + Creditcoin
CC3)

38

Hardhat contract tests
(executor, registry, verifier
adapter)

115 / 115

end-to-end checks passing
against live production

10

adversarial tamper scenarios
verified against the live
contract

One production policy has genuinely completed its full lifecycle — Safe approval, Attestcoin attestation, on-chain verification, and a real 2,500 MockUSDC payout — each step backed by a confirmed transaction hash on both chains.

Three real bugs were found and fixed during end-to-end testing (response-shape mismatches causing 500s, and a React infinite-render-loop crash after selecting a Safe) — documented with root cause and fix in docs/DEMO_GUIDE.md, rather than hidden.

Built on production-grade tooling, end to end

LAYER	STACK
Contracts	Solidity 0.8.28, Hardhat, OpenZeppelin 5, @gluwa/asc-contracts
Backend	Fastify 5, Prisma + Postgres, ethers v6, @gluwa/usc-sdk, pino
Frontend	Next.js 16, React 19, wagmi 3 / viem 2, @safe-global/protocol-kit + api-kit, TanStack Query, Tailwind 4
Infra	Railway (Postgres + backend + frontend services), npm workspaces monorepo

Deployed contracts

PolicyRegistry (Sepolia):
0xEE1920869bFCB8b24C1b5AAE45927E7ad12f07F0
Authority Safe (Sepolia):
0xf4b6a941B0bB4699cba45970cBf7c476ccE49dDd
SafeRootPolicyExecutor (CC3):
0x57dE3c48b747A812fD643900568EA9FbCE6dF7cB
AttestcoinVerifierAdapter (CC3):
0x8964F00624b3aAF3f74d51db2016571dC398374d

Live services

Frontend: frontend-production-78b3.up.railway.app
Backend API: backend-production-1ff6f.up.railway.app
Repository: github.com/Vamp-Labs/Saferoot

Beyond the hackathon

- **Guardian rotation & multi-guardian quorum** — today a single guardian address can pause; a natural extension is a small guardian committee with its own threshold.
- **Policy templates as a public registry** — let any Creditcoin protocol publish an "integration" (target + allowed selectors) so more DAOs can point a policy at it without custom backend wiring.
- **Batch policies** — one Safe approval authorizing a sequence of dependent actions across multiple Creditcoin contracts, not just parallel independent actions.
- **Mainnet hardening** — formal audit of SafeRootPolicyExecutor's replay/expiry/reentrancy boundaries before any real-value mainnet deployment.

THANK YOU

SafeRoot Policy

One approval. A trustless proof. A contract that only does exactly what was signed for.

Live app: frontend-production-78b3.up.railway.app

Repository: github.com/Vamp-Labs/Saferoot

Demo video: youtu.be/6N_fVw4CcgM

Full docs: [docs/README.md](#) · [docs/HACKATHON_SUBMISSION.md](#) · [docs/DEMO_GUIDE.md](#)